InfraMind: Automated Incident Detection, Root Cause Analysis & Alerting System

Product Requirements Document (PRD) | ***Draft version***

Document Version	**v1.1**
Date	June 2026
Status	Concept / Pre-Development
Target Stack	Kubernetes/Containers, Cloud (AWS/GCP/Azure), Microservices, Traditional Servers/VMs
Data Sources	Logs (ELK/Loki), Metrics (Prometheus/Datadog), Traces (Jaeger/OpenTelemetry), Alerts (PagerDuty/Grafana)
AI Capabilities	Incident Detection + Root Cause Analysis (RCA)
Classification	Research Confidential

1. Executive Summary

InfraMind is an AI-powered observability and incident intelligence system designed to autonomously detect incidents, analyze root causes, and deliver actionable alerts across heterogeneous infrastructure environments. It targets organizations running workloads across Kubernetes clusters, cloud platforms (AWS, GCP, Azure), microservices architectures, and traditional server fleets, the full spectrum of modern hybrid infrastructure.

Unlike conventional monitoring tools that emit raw metric or log alerts, InfraMind ingests and correlates signals across four data source categories logs, metrics, distributed traces, and existing alert streams to build a holistic picture of system health. When an anomaly is detected, InfraMind's Root Cause Analysis (RCA) engine traverses the correlated signal graph to identify the most probable origin of the incident, and generates a context-rich alert that tells on-call engineers not just that something is wrong, but why.

This document defines the product vision, functional and non-functional requirements, proposed system architecture, research roadmap, and success criteria for the InfraMind research project.

2. Problem Statement

2.1 The Challenge

Modern infrastructure spans Kubernetes pods, cloud-managed services, legacy VMs, and hundreds of microservices generating a torrent of observability signals from logs, metrics, traces, and alerting tools. Current incident management approaches suffer from four critical failures:

- Alert fatigue: Raw alerts from Grafana, PagerDuty, and Datadog are high-volume and low-fidelity. On-call engineers are desensitized by noise.
- Fragmented visibility: Logs, metrics, and traces live in separate tools with no automated correlation. Engineers manually stitch signals together during incidents.
- No root cause context: Existing tools tell you a service is down but not why. Root cause investigation is manual, slow, and expertise-dependent.
- Infrastructure heterogeneity: Tools tuned for Kubernetes often miss VM-level failures; cloud-native monitors miss on-prem signals. Coverage gaps exist.

2.2 Target Users

- Site Reliability Engineers (SREs) and Platform Engineers managing multi-stack environments
- On-call engineers who need fast, accurate, root-cause-aware incident notifications
- Engineering leads and managers needing visibility into infrastructure health and incident trends

2.3 Research Hypothesis

By ingesting and correlating logs, metrics, distributed traces, and upstream alerts from a heterogeneous infrastructure stack, InfraMind can reduce Mean Time to Detect (MTTD) by at least 50%, reduce alert noise by 60%, and provide automated Root Cause Analysis (RCA) that reduces Mean Time to Resolve (MTTR) by at least 40%.

3. Product Vision & Goals

3.1 Vision Statement

To build an intelligent infrastructure mind that watches every layer of a system from Kubernetes pods to cloud services to bare-metal servers correlates signals across all observability pillars, determines the root cause of failures autonomously, and surfaces that intelligence to engineering teams before users feel the impact.

3.2 Research Goals

- Design and validate a unified signal ingestion pipeline that supports logs (ELK/Loki), metrics (Prometheus/Datadog), traces (Jaeger/OpenTelemetry), and alerts (PagerDuty/Grafana) across all four infrastructure target stacks.
- Develop a cross-signal correlation engine that links anomalies from different observability pillars to identify compound incidents spanning multiple infrastructure layers.
- Research and prototype an automated Root Cause Analysis (RCA) engine that traverses correlated signal graphs to identify the most probable origin of an incident.
- Evaluate AI/ML approaches for anomaly detection and RCA including LLM-based reasoning, graph-based causal analysis, and classical statistical models.
- Demonstrate a significant reduction in alert noise and MTTD compared to using individual monitoring tools in isolation.

3.3 Out of Scope (v1 Research Phase)

- Automated remediation or auto-healing (planned for future research phase)
- Full production deployment this phase is research and prototype only
- Custom APM agent development **InfraMind** consumes from existing instrumentation

4. Functional Requirements

Requirements are classified as High (must have for research validation), Medium (important but deferrable), or Low (nice to have). Status: all Not Started as of v1.1.

ID	Requirement	Priority	Status
4.1 Data Ingestion — Logs			

ID	Requirement	Priority	Status
FR-01	Ingest structured and unstructured logs from ELK (Elasticsearch/Logstash/Kibana) via API.	**High**	Not Started
FR-02	Ingest log streams from Loki using the HTTP push/query API.	**High**	Not Started
FR-03	Parse log entries to extract: service name, log level, error codes, timestamps, and trace IDs.	**High**	Not Started
FR-04	Support log ingestion from all four infrastructure stacks: Kubernetes pod logs, cloud service logs (CloudWatch, GCP Logging), microservice application logs, and syslog from VMs.	**High**	Not Started
4.2 Data Ingestion — Metrics			
FR-05	Scrape metrics from Prometheus endpoints at configurable intervals (default: 30s).	**High**	Not Started
FR-06	Poll Datadog metrics API for service-level and infrastructure-level metrics.	**High**	Not Started
FR-07	Collect Kubernetes-specific metrics: pod CPU/memory, node pressure, container restarts, OOMKill events.	**High**	Not Started
FR-08	Collect cloud provider metrics: AWS CloudWatch, GCP Monitoring, Azure Monitor.	**Medium**	Not Started
FR-09	Collect VM/host-level metrics: CPU, memory, disk I/O, network saturation.	**Medium**	Not Started
4.3 Data Ingestion — Traces			
FR-10	Ingest distributed traces from Jaeger via its Query API or trace streaming endpoint.	**High**	Not Started
FR-11	Ingest trace data from OpenTelemetry Collector via OTLP (gRPC or HTTP).	**High**	Not Started
FR-12	Extract span-level data: service name, operation, duration, error status, parent/child relationships.	**High**	Not Started
FR-13	Build service dependency graphs from trace data to support RCA traversal.	**High**	Not Started
4.4 Data Ingestion — Alerts			
FR-14	Consume incoming alert payloads from PagerDuty via webhook integration.	**High**	Not Started
FR-15	Consume Grafana alerting rule triggers via webhook or API.	**High**	Not Started
FR-16	Normalize alert payloads from all sources into a unified internal schema (service, severity, description, timestamp).	**High**	Not Started
FR-17	Handle back-pressure and buffering during high-volume alert spikes.	**Medium**	Not Started

ID	Requirement	Priority	Status
4.5 Incident Detection			
FR-18	Detect metric anomalies using statistical baselines (z-score, EWMA) per service and per infrastructure layer.	**High**	Not Started
FR-19	Detect log anomaly patterns: sudden error rate increase, repeated exception types, log volume spikes.	**High**	Not Started
FR-20	Detect trace anomalies: elevated span error rates, latency regressions, broken dependency calls.	**High**	Not Started
FR-21	Correlate signals across all four pillars (logs, metrics, traces, alerts) within a configurable time window.	**High**	Not Started
FR-22	Classify detected incidents into severity: Critical, Warning, Info.	**High**	Not Started
FR-23	Suppress duplicate/redundant alerts for the same underlying incident (deduplication).	**Medium**	Not Started
FR-24	Support user-defined detection rules as a complement to AI-driven detection.	**Low**	Not Started
4.6 Root Cause Analysis (RCA)			
FR-25	Upon incident detection, traverse the service dependency graph (built from traces) to identify upstream failure origins.	**High**	Not Started
FR-26	Correlate detected anomalies with recent deployment events, config changes, or resource exhaustion patterns.	**High**	Not Started
FR-27	Generate a ranked list of probable root causes with supporting evidence from each signal pillar.	**High**	Not Started
FR-28	Use LLM-based reasoning to produce a natural-language RCA summary: what failed, why, and which services are affected.	**Medium**	Not Started
FR-29	Estimate blast radius: list of services and infrastructure components likely impacted by the root cause.	**Medium**	Not Started
FR-30	Store RCA results alongside incident records for retrospective analysis and model improvement.	**Medium**	Not Started
4.7 Alerting & Notification			
FR-31	Emit structured incident alerts containing: title, severity, affected services, detected signals summary, RCA result, timestamp.	**High**	Not Started
FR-32	Deliver alerts via at least two notification channels (Slack + webhook minimum).	**High**	Not Started
FR-33	Support severity-based alert routing (e.g. Critical to PagerDuty, Warning to Slack).	**Medium**	Not Started
FR-34	Include AI-generated RCA summary and blast radius in the alert payload.	**Medium**	Not Started

ID	Requirement	Priority	Status
FR-35	Support alert suppression / maintenance windows to reduce noise during planned changes.	Low	Not Started
4.8 Agent Observability & Feedback			
FR-36	Log all detection and RCA decisions with supporting signals for audit and reproducibility.	High	Not Started
FR-37	Expose agent health metrics: ingestion lag, detection latency, alert throughput.	Medium	Not Started
FR-38	Provide a feedback interface to label false positives and false negatives for model evaluation.	Low	Not Started

5. Proposed System Architecture

InfraMind is structured as a five-stage pipeline. Each stage is independently deployable and designed for extensibility.

5.1 Ingestion Layer

- Log Collector: Pulls from ELK and Loki APIs. Supports Kubernetes pod logs, cloud service logs (CloudWatch, GCP Logging, Azure Monitor), microservice app logs, and VM syslog. Parses and enriches with service metadata.
- Metrics Collector: Scrapes Prometheus endpoints and polls Datadog API. Collects Kubernetes metrics (kube-state-metrics, cAdvisor), cloud provider metrics, and host-level metrics from VM node exporters.
- Trace Ingester: Consumes distributed traces from Jaeger Query API and OpenTelemetry Collector (OTLP). Extracts service topology and span-level error data. Builds and continuously updates the service dependency graph.
- Alert Consumer: Receives webhook payloads from PagerDuty and Grafana. Normalizes to a unified internal alert schema.
- Event Bus: All signals published to an internal event bus (Kafka or Redis Streams) for decoupled downstream processing and replay.

5.2 Correlation & Detection Engine

- Signal Normalizer: Enriches all signals with a common service identifier, infrastructure layer tag (k8s / cloud / microservice / vm), and wall-clock timestamp.
- Time-Window Correlator: Groups signals from logs, metrics, traces, and alerts that share the same service or occur within a configurable time window (default: 5 minutes).
- Anomaly Detector: Applies per-signal-type statistical models EWMA baselines for metrics, log pattern frequency analysis for logs, span latency regression for traces.
- Incident Assembler: Combines correlated anomalous signals into a single Incident object with a unique ID, severity score, and list of contributing signals.

5.3 Root Cause Analysis (RCA) Engine

- Dependency Graph Traversal: Uses the service topology graph (derived from traces) to walk upstream from the affected service, identifying which dependency most likely originated the failure.
- Change Correlation: Cross-references incident timing with deployment events, Kubernetes rollouts, config map changes, or cloud provider incident feeds.
- Evidence Aggregator: Collects supporting evidence from each signal pillar (specific log lines, metric charts, anomalous spans, triggering alert) into a structured RCA evidence bundle.
- AI Reasoning Module: Optionally invokes an LLM to synthesize the evidence bundle into a natural-language RCA summary, probable root cause ranking, and blast radius assessment.

5.4 Alerting Layer

- Deduplication Engine: Suppresses redundant alerts for the same incident within a cooldown window.
- Severity Router: Routes alerts to configured channels based on severity (Critical, Warning, Info).
- Alert Payload Builder: Assembles the final alert object title, severity, affected services, RCA summary, blast radius, contributing signals, and runbook link.
- Notification Dispatchers: Pluggable adapters for Slack, PagerDuty, email, and generic webhooks.

5.5 Storage & Audit Layer

- Incident Store: Persists all detected incidents, their contributing signals, RCA results, and resolution status.
- Signal Archive: Time-series store for raw ingested signals (used for retrospective analysis and model training).
- Audit Log: Immutable log of all detection and RCA decisions for reproducibility and evaluation.

6. Non-Functional Requirements

6.1 Performance

- End-to-end detection latency (ingestion to alert emission): < 60 seconds for Critical incidents.
- RCA generation latency: < 120 seconds from incident detection (can be async alert emitted first, RCA appended).
- Ingest throughput: up to 10,000 log events/sec, 1,000 metric data points/sec, 5,000 spans/sec in the prototype environment.

6.2 Reliability

- Loss of any single data source must not crash the pipeline graceful degradation required.
- At-least-once delivery guaranteed for all generated alerts.
- The event bus provides buffering against downstream processing delays.

6.3 Accuracy (Research Success Criteria)

- Alert Precision: > 80% of emitted alerts correspond to real incidents.
- Alert Recall: > 75% of injected faults detected within the target window.
- RCA Accuracy: > 70% of root cause identifications match the ground truth (fault injection label).
- Alert Noise Reduction: > 60% fewer alerts vs. raw tool baseline.

6.4 Extensibility

- All data source connectors must be modular adapters adding a new source requires only implementing a standard ingestion interface.
- Detection models and RCA strategies must be swappable via configuration.
- Infrastructure stack support must be additive new stack types (e.g. serverless) can be added without re-architecting the pipeline.

7. Research Roadmap & Milestones

Phase	Milestone	Duration	Deliverable
Phase 1	Literature review, architecture design, tech stack selection	3 weeks	Architecture doc, ADRs, tech stack decision
Phase 2	Ingestion layer: logs (ELK + Loki) + metrics (Prometheus + Datadog)	4 weeks	Working collectors, unified event bus
Phase 3	Ingestion layer: traces (Jaeger + OTel) + alerts (PagerDuty + Grafana)	3 weeks	Trace ingester, service dependency graph, alert normalizer
Phase 4	Signal correlation & anomaly detection engine	5 weeks	Cross-pillar correlation, incident assembler, evaluation on injected faults
Phase 5	Root Cause Analysis engine (graph traversal + AI reasoning)	5 weeks	RCA engine, LLM integration, RCA accuracy evaluation
Phase 6	Alerting pipeline, deduplication, notification dispatchers	2 weeks	End-to-end alert delivery, Slack + webhook output
Phase 7	Full system evaluation & research paper	4 weeks	Precision/recall/RCA results, research paper draft

8. Risks & Mitigations

8.1 Technical Risks

- Risk: Cross-pillar signal correlation generates too many false positives from coincident but unrelated anomalies across the four infrastructure stacks. Mitigation: Use infrastructure layer tagging and the service dependency graph as correlation constraints only correlate signals from services with a known dependency relationship.
- Risk: LLM-based RCA reasoning adds latency that breaches the 120-second RCA target. Mitigation: Run LLM reasoning asynchronously emit the structured alert immediately, append the AI RCA summary as a follow-up message within the SLA window.
- Risk: API rate limits from Datadog, PagerDuty, or cloud metric APIs throttle ingestion during high-load periods. Mitigation: Implement adaptive polling with exponential backoff and local caching of last known signal values.
- Risk: Distributed trace coverage is incomplete; not all services are instrumented with Jaeger or OpenTelemetry. Mitigation: RCA engine must gracefully degrade to log and metric-based root cause inference when trace data is absent.

8.2 Research Risks

- Risk: Insufficient labeled incident data to evaluate RCA accuracy. Mitigation: Use fault injection (Chaos Mesh for Kubernetes, AWS Fault Injection Simulator for cloud) to generate ground-truth labeled incidents across all four stack types.
- Risk: Results specific to the test environment may not generalize to production-scale systems. Mitigation: Use widely adopted open-source reference architectures (Google Online Boutique for microservices/K8s, a cloud-deployed variant for AWS/GCP testing) as the evaluation environment.

9. Open Questions & Decisions Needed

- AI/ML approach for RCA: LLM-based reasoning (richer context, higher cost/latency) vs. graph-based causal inference (deterministic, faster) vs. hybrid?
- Event streaming backbone: Apache Kafka (production-grade, higher operational overhead) vs. Redis Streams (simpler, lower throughput ceiling)?
- LLM provider for RCA summarization: Claude API, OpenAI GPT-4, or a self-hosted open-source model (Llama 3)?
- Service dependency graph: static (seeded from config) or dynamic (continuously updated from trace data)? How to handle services with no trace instrumentation?
- Cloud metrics strategy: deploy InfraMind inside each cloud account (sidecar model) or use cross-account metric APIs from a central collector?
- Fault injection framework: Chaos Mesh (K8s-native) + AWS FIS + manual VM fault injection how to unify ground-truth labeling across all four stacks?

10. Success Metrics

The InfraMind research project will be considered successful if the prototype demonstrates the following outcomes across the multi-stack test environment:

Metric	Target	Measurement Method
Mean Time to Detect (MTTD)	< 60 seconds	Fault injection timestamp vs. first alert
Alert Precision	> 80%	Manual labeling of emitted alerts
Alert Recall	> 75%	Detected vs. total injected faults
Alert Noise Reduction	> 60% vs. baseline	InfraMind alert count vs. raw tool count
RCA Accuracy	> 70%	Identified root cause vs. injected fault ground truth
RCA Latency	< 120 seconds	Detection timestamp vs. RCA delivery
False Positive Rate	< 20%	Alerts with no corresponding injected fault
Cross-stack Coverage	All 4 stacks	Validated fault injection across K8s, Cloud, Microservices, VMs

11. Appendix: Glossary

- **InfraMind** The Automated incident detection, RCA, and alerting system defined in this PRD.
- **SRE** Site Reliability Engineering: applying software engineering to infrastructure and operations.
- **MTTD** Mean Time to Detect: time from incident start to first alert.
- **MTTR** Mean Time to Resolve: time from incident detection to full resolution.
- **RCA** Root Cause Analysis: identifying the underlying cause of an incident, not just its symptoms.
- **ELK** Elasticsearch, Logstash, Kibana: an open-source log management stack.
- **Loki** A horizontally scalable log aggregation system by Grafana Labs.
- **Prometheus** An open-source systems monitoring and alerting toolkit.
- **Datadog** A cloud-scale monitoring and analytics SaaS platform.
- **Jaeger** An open-source distributed tracing system originally built by Uber.
- **OpenTelemetry** A vendor-neutral observability framework for generating and collecting traces, metrics, and logs.
- **OLTP** OpenTelemetry Protocol: the standard wire protocol for telemetry data.
- **PagerDuty** An incident management and on-call alerting platform.
- **Grafana** An open-source analytics and monitoring visualization platform.
- **Chaos Mesh** A cloud-native chaos engineering platform for Kubernetes.
- **AWS FIS** AWS Fault Injection Simulator: a managed chaos engineering service for AWS.

- **Service Dependency Graph** A directed graph representing which services call which other services, derived from distributed trace data.
- **LLM** Large Language Model: a deep learning model trained on large text corpora for natural language understanding and generation.
- **Signal Correlation** Linking related anomalies across different observability pillars that share a common root cause.
- **Blast Radius** The set of services and infrastructure components likely impacted by a given root cause.

— End of Document | InfraMind PRD v1.0 —